



Aprofundamento Teórico

Do Modelo Relacional ao NoSQL: fundamentos, formalizações e contexto histórico

Aula complementar de contextualização teórica

Objetivos de aprendizagem



Contexto histórico

Da era hierárquica ao NoSQL



Fundamentos relacionais

Codd, álgebra relacional e seus limites



Limitações e escala

Rigidez, escalabilidade, JOINS



ACID x BASE

Garantias transacionais formalizadas



Teorema CAP e PACELC

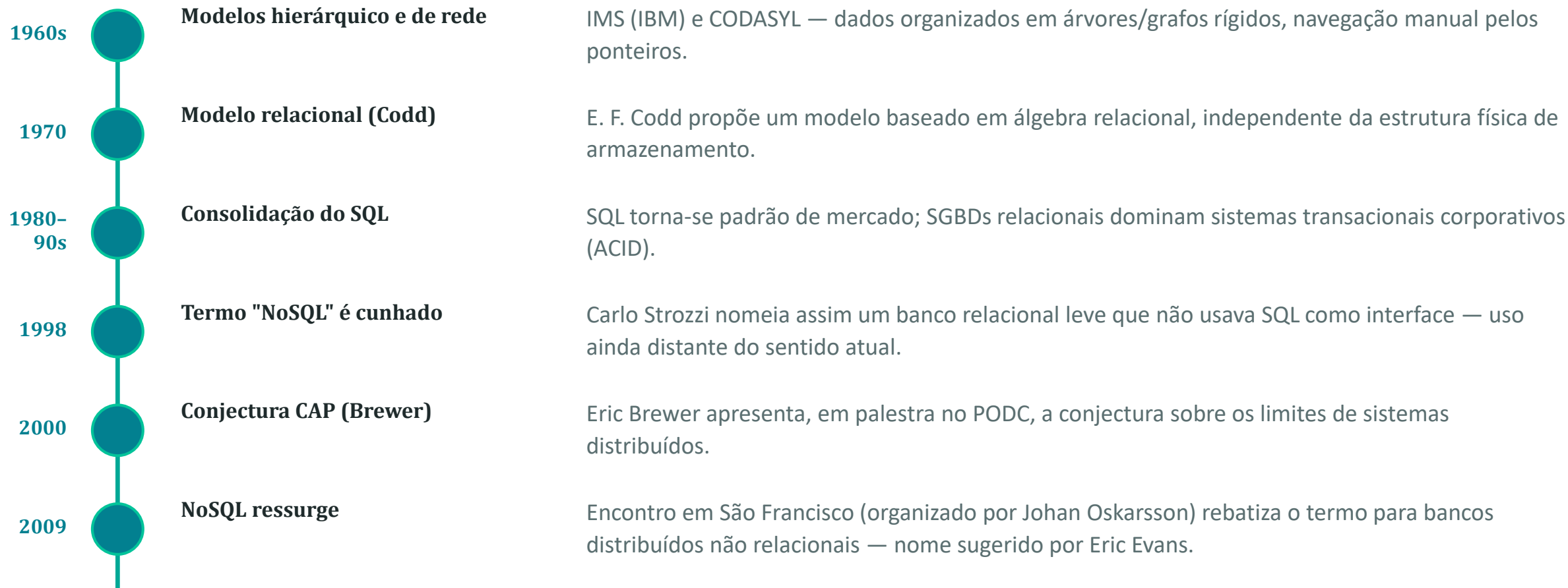
Formalização e extensão acadêmica



Síntese e discussão

NoSQL, referências e debate aberto

Linha do tempo dos modelos de banco de dados



O modelo relacional: base formal (Codd, 1970)



Edgar F. Codd, “A Relational Model of Data for Large Shared Data Banks”, Communications of the ACM, 1970 — artigo fundador que propôs representar dados em relações (tabelas) matemáticas, manipuláveis por álgebra relacional, independentes de como são fisicamente armazenados.

Relação (tabela)

Conjunto de tuplas (linhas) com atributos (colunas) tipados — base da organização de dados.

Álgebra relacional

Conjunto de operações formais (seleção, projeção, junção, união) que fundamentam o SQL.

Chaves e integridade

Chave primária/estrangeira garantem unicidade e consistência referencial entre relações.

Formas normais

Regras de normalização (1FN, 2FN, 3FN) reduzem redundância e anomalias de atualização.

Esse formalismo é o que torna o relacional tão robusto para consistência forte — e, como veremos, também a raiz de suas limitações em escala.

Estruturados, semiestruturados e não estruturados

Estruturados

Esquema fixo, tipos definidos previamente. Ex.: tabela relacional de clientes.

Semiestruturados

Alguma organização (tags, hierarquia), mas sem esquema rígido. Ex.: JSON, XML.

Não estruturados

Sem modelo predefinido. Ex.: imagens, vídeos, texto livre, posts.



Dado de mercado: estimativas de institutos como IDC e Gartner indicam que 80–90% de todo o dado gerado atualmente é não estruturado — motivador direto da adoção de bancos NoSQL orientados a documentos e a outras formas flexíveis de armazenamento.

Big Data: os 5 V's e sua origem acadêmica

Origem do conceito: Doug Laney (Gartner/META Group, 2001) definiu inicialmente 3 dimensões — Volume, Velocidade e Variedade — em relatório sobre gestão de dados. Veracidade e Valor foram incorporados posteriormente pela indústria (IBM e outros), consolidando o modelo dos "5 V's".

Volume

Crescimento exponencial de dados gerados (redes sociais, IoT, transações).

Velocidade

Dados gerados e processados quase em tempo real (streaming, sensores).

Variedade

Múltiplos formatos e fontes coexistindo — estruturado, semi e não estruturado.

Veracidade

Qualidade e confiabilidade do dado nem sempre garantidas na origem.

Valor

O dado só é útil se transformado em informação/decisão acionável.

Extensões acadêmicas posteriores chegam a propor até 7 V's (incluindo Variabilidade e Visualização) — o essencial para a disciplina é entender que esses fatores, em conjunto, ultrapassam a capacidade prática dos SGBDs relacionais tradicionais.

Limitações do modelo relacional



Rigidez de esquema

Schema-on-write: alterar tabelas em produção é custoso e arriscado em bases grandes.



Escala vertical x horizontal

Escalar "para cima" tem limite físico e custo não linear; NoSQL nasce pensado para escalar "para os lados".



Custo de JOINs distribuídos

JOINs entre tabelas grandes ficam caros conforme volume e distribuição entre nós crescem.



Impedância objeto-relacional

Objetos aninhados da aplicação exigem múltiplas tabelas e camadas de ORM para reconstrução.



Dados fortemente conectados

Relacionamentos multidirecionais e profundos são caros em JOINs recursivos no relacional.



ACID x BASE

Consistência forte tradicional x disponibilidade/desempenho com consistência eventual.

Escalabilidade: vertical x horizontal

Vertical (scale-up)

Aumenta a capacidade de uma única máquina (CPU, RAM, disco). Curva de custo não linear: dobrar a capacidade não custa o dobro — custa proporcionalmente mais. Existe um teto físico de hardware.

Horizontal (scale-out)

Distribui dados entre vários nós de menor capacidade, somando recursos. Cresce "infinitamente" adicionando máquinas — abordagem nativa da maioria dos bancos NoSQL, geralmente com particionamento (sharding) automatizado.



Por que isso importa na prática

- Sharding manual em SGBDs relacionais exige planejamento cuidadoso da equipe de banco de dados.
- Bancos NoSQL como Cassandra distribuem dados automaticamente por um anel de nós (consistent hashing).
- Escalar horizontalmente introduz novos desafios: replicação, consistência entre réplicas e latência de rede — temas que o Teorema CAP formaliza.

ACID



Atomicidade

Uma transação é "tudo ou nada": se uma parte falha, toda a transação é revertida (rollback).



Consistência

A transação leva o banco de um estado válido a outro, respeitando todas as regras de integridade.



Isolamento

Transações concorrentes não interferem entre si — o resultado é como se fossem executadas em série.



Durabilidade

Uma vez confirmada (commit), a alteração persiste mesmo diante de falhas do sistema.

BASE: a alternativa adotada por muitos NoSQL

BA

Basically Available

O sistema garante disponibilidade na maior parte do tempo, mesmo diante de falhas parciais.

S

Soft state

O estado do sistema pode mudar ao longo do tempo, mesmo sem novas entradas, devido à consistência eventual.

EC

Eventually consistent

Réplicas convergem para o mesmo valor com o tempo, mas podem divergir temporariamente.

BASE não é "pior" que ACID — é um trade-off deliberado: abre mão de consistência imediata em troca de disponibilidade e desempenho em escala, quando o cenário tolera isso.

Teorema CAP

2000

Eric Brewer (UC Berkeley)

Apresenta a conjectura em palestra convidada no ACM PODC (Symposium on Principles of Distributed Computing): impossível garantir Consistência, Disponibilidade e Tolerância a Partição simultaneamente.

2002

Seth Gilbert & Nancy Lynch (MIT)

Publicam a prova formal da conjectura de Brewer no ACM SIGACT News, demonstrando-a para redes assíncronas e parcialmente síncronas.

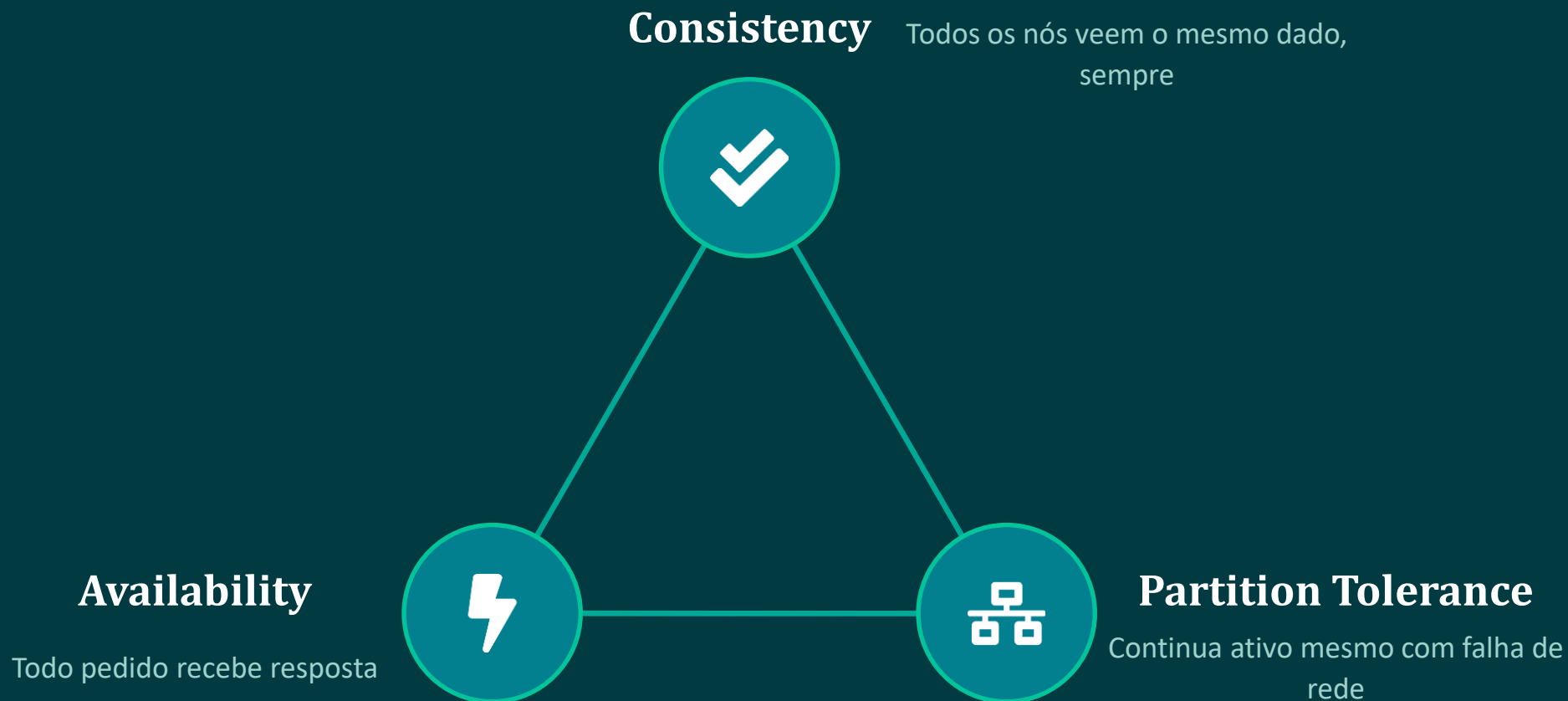
2012

Eric Brewer revisita o tema

Em artigo retrospectivo, esclarece que a escolha C x A não é binária: partições são raras e o trade-off pode ser gerenciado dinamicamente.

O triângulo do Teorema CAP

Diante de uma partição de rede, um sistema distribuído precisa escolher entre Consistência e Disponibilidade.



O modelo PACELC (Abadi, 2012)



Daniel Abadi (Yale), “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story”, IEEE Computer, 2012.

O argumento central de Abadi: partições de rede são eventos raros. Na maior parte do tempo, o sistema está conectado ("Else") — e mesmo assim existe um trade-off entre Latência e Consistência que o CAP não descreve.

PAC — se Partição ocorrer

Trade-off entre Availability (A) e Consistency (C) — exatamente o CAP tradicional.

ELC — mesmo sem partição (Else)

Trade-off entre Latency (L) e Consistency (C) — replicar dados para reduzir latência custa consistência imediata.

Sistemas reais posicionados no espectro CAP / PACELC

Sistema	Categoria	Prioridade (partição)	Prioridade (sem partição)
PostgreSQL (replicado)	Relacional distribuído	CP	EC (consistência)
MongoDB	Documentos	CP (por padrão)	EC / ajustável
Cassandra	Colunas	AP (ajustável)	EL (latência)
Redis (cluster)	Chave-valor	AP	EL (latência)
Neo4j (single-instance)	Grafos	CP	EC (consistência)
DynamoDB	Chave-valor gerenciado	AP (ajustável)	EL / EC (configurável)

Classificações simplificadas para fins didáticos — configurações específicas (nível de consistência, replicação) podem alterar o posicionamento real de cada sistema.

O termo "NoSQL"

1998 — primeira aparição

Carlo Strozzi nomeia "NoSQL" um banco de dados relacional leve, de código aberto, que simplesmente não usava SQL como linguagem de interface — o sentido é distinto do uso atual, e o banco continuava sendo relacional.

2009 — o sentido atual

Johan Oskarsson organiza um encontro em São Francisco para discutir bancos de dados distribuídos e não relacionais; Eric Evans sugere reaproveitar o termo "NoSQL" para esse novo movimento — popularizando o sentido usado até hoje: "Not Only SQL".

Panorama das quatro categorias NoSQL

Categoria	Estrutura de dado	Ponto forte	Ferramenta usada no semestre
Chave-valor	Par chave → valor	Latência mínima	Redis
Documentos	JSON/BSON aninhado	Flexibilidade de esquema	MongoDB
Colunas	Famílias de colunas distribuídas	Altíssima escrita distribuída	Cassandra
Grafos	Nós e arestas	Relacionamento profundo	Neo4j

O aprofundamento prático de cada categoria — com hands-on nas ferramentas gratuitas — acontece nos encontros seguintes do cronograma.

Referências bibliográficas

- CODD, E. F. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 1970.
- GILBERT, S.; LYNCH, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, v. 33, 2002.
- ABADI, D. Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story. IEEE Computer, v. 45, n. 2, 2012.
- SADALAGE, P.; FOWLER, M. NoSQL Essencial. Novatec, 2013.
- BREWER, E. Towards Robust Distributed Systems. Keynote, ACM PODC, 2000.
- BREWER, E. CAP Twelve Years Later: How the "Rules" Have Changed. IEEE Computer, 2012.
- LANEY, D. 3D Data Management: Controlling Data Volume, Velocity and Variety. META Group (Gartner), 2001.
- ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados: Fundamentos e Aplicações. 7ed. Pearson, 2019.



Discussão

1 Um sistema que você usa no dia a dia prioriza consistência ou disponibilidade? Como você percebe isso?

2 Em que cenário PACELC explica melhor uma escolha de arquitetura do que o CAP sozinho?

3 Se o modelo relacional é tão bem formalizado (Codd, 1970), por que ele ainda domina sistemas transacionais críticos?